

5G Network Slice Admission Control Using Optimization and Reinforcement Learning

Md Ariful Haque[†], Vassilka Kirova[‡]

[†]*Computational Modeling and Simulation Engineering, Old Dominion University, Norfolk, VA, USA*

[‡]*Nokia Bell Labs Consulting, Murray Hill, NJ, USA*

mhaqu001@odu.edu[†], vassilka.kirova@bell-labs-consulting.com[‡]

Abstract—Network slicing can enable operators to maximize revenue by optimizing the utilization of network resources. As a result, slice admission control and resource allocation decisions become critical tasks for 5G network orchestration. In this work, we examine two complimentary approaches that address the challenge of slice admission control: (1) dynamically assessing and accepting a set of slices that generate greater revenue using a queue-based optimization algorithm, and (2) automating the admission decision using reinforcement learning (RL). The simulation results from the first approach show that the proposed optimization algorithm always delivers higher revenue than accepting the slices in any other order. In the second approach, we utilize several reinforcement learning algorithms to train an agent (orchestrator) to dynamically make decisions based on the environment. Our results show that the Deep Q Network (DQN) algorithm provides higher cumulative rewards (i.e., cumulative revenue) than the other tested algorithms (SARSA, Expected SARSA, and Q-Learning). The two approaches are complementary, and can be deployed either separately or simultaneously to maximize network operator's revenue.

Index Terms—5G, Network Slicing, Admission Control, Optimization, Reinforcement Learning, DQN.

I. INTRODUCTION

Network slicing has a considerable potential in effective resource allocation and customization of the Fifth Generation (5G) mobile networks. In fact, it is one of the prominent features of 5G to allow connectivity and data processing tailored to customers' requirements. Essentially, a network slice is an end-to-end logical network instance that runs on a shared physical infrastructure and can provide quality of service (QoS) that is agreed on between a provider and a client. The concepts of network slicing and NSaaS (Network Slice as a Service) are standardized by 3GPP technical specifications 23.501 [1]. Network slicing can provide significant improvements in scalability, flexibility, and profitability to mobile network operators (MNO) [2].

To benefit from the high market potential of slicing, infrastructure providers (i.e., MNO) still need to address the challenge of introducing effective slice admission control mechanisms to maximize their return on investment. Furthermore, because of the enormous number of slices to be processed simultaneously, there is a need to develop autonomous mechanisms that an orchestrator can use to process the slice requests and make admission decisions dynamically in a resource constrained environment. One additional challenge

that MNOs face is how to reduce the opportunity cost while taking slice admission or rejection decisions.

Typically, there are two contexts in which slice admissions are managed. First, an MNO provides services directly to end customers and operates the slices via its orchestrator, for example, the work of Han et al. [3]. Second is the resource-sharing case of NSaaS, where an MNO partitions its resources into smaller logical units (i.e., slices) and leases them to external tenants such as mobile virtual network operators (MVNOs), thus getting revenue for rendering slice services. For example, the work of Bakri et al. [4] addresses the slice admission from the second perspective. In both cases, the MNO seeks to maximize the total revenue by allocating resources having constraints imposed by the limitations of the resource pool. In this work, we are focusing on slice admission control from the second context's perspective.

Due to recent advances in artificial intelligence (AI), several AI-based techniques are gaining attention from the research community to handle the massive slice requests autonomously. For example, Han et al. [5] proposed a 5G architecture to control admission and congestion in 5G network slicing using Q-learning. Bakri et al. [4] present several reinforcement learning-based algorithms to determine the slices with higher rewards (i.e., higher revenues). In our work, we use an approach similar to the one presented by Bakri et al. [4]. However, Bakri's work does not consider any lost opportunities due to slice rejection, while we include the opportunity cost in the models.

This work addresses two specific challenges related to the slice admission control problem. These are (1) dynamically accepting slices that generate maximum revenue and minimize the opportunity cost, considering capacity constraints and slice priority (i.e., serving urgency), and (2) automating the admission decision to handle massive slice requests without manual intervention. We first propose a discrete event queue-based model to accept slices. Then, we formulate the slice admission as an optimization problem that maximizes the revenues and minimizes the opportunity cost due to slice non-acceptance. We also consider the priority of the slice requests using a priority-based scheduling scheme with aging by updating the priority values so that an agreed-on slice request does not go unserved while accepting/serving more profitable slice requests. Finally, we have formulated a Markov decision process (MDP)-based slice model and utilized rein-

forcement learning algorithms to automate the slice admission process. We simulated and compared the performance of several algorithms such as SARSA, Expected SARSA, Q-Learning, and Deep Q-Network (DQN) and found that the DQN provides a higher long-term cumulative reward for the infrastructure provider or MNOs. The contributions of this work are:

- A novel queue-based discrete event modeling and multi-objective optimization of slice admission control for maximizing revenue and minimizing opportunity cost
- A MDP-based slice model for handling slice requests autonomously using reinforcement learning techniques.

We organize the rest of the paper in the following manner. We present the slice admission optimization algorithm in Section II. In Section III, we illustrate the setup and analysis of simulation results for the slice optimization process. In Section IV we discuss the reinforcement learning model for dynamically handling the slice admission process. Section V presents the simulation results for the reinforcement learning-based admission control model. Finally, section VI concludes the paper by discussing the essential findings and future directions.

II. SLICE ADMISSION OPTIMIZATION MODEL

There are two key players in the optimization model. They are the infrastructure provider (i.e., MNO denoted as ‘InfP’) and the tenants (i.e., MVNOs). Tenants request logical resources from the InfP. InfP receives resource allocation requests (i.e., slice requests) from multiple tenants (also a tenant can request different types of resources as needed). Then, an optimization process is applied to determine the most beneficial slice requests to serve. This work considers resource limitations on the downlink (DL) and uplink (UL) capacity to generate realistic scenarios. We can extend the resource limitations to other types of resources by incorporating additional model parameters.

A. Slice Requests and Slice Pipeline

We define a slice request by a tuple as follows, $S_{req}^{(i)} = (S_{id}^{(i)}, S_{type}^{(i)}, S_{DLreq}^{(i)}, S_{ULreq}^{(i)}, S_{Nreq}^{(i)}, S_{STreq}^{(i)}, S_{priority}^{(i)})$. Here, $S_{req}^{(i)}$ indicates slice request i . Each slice request has a system-generated identifier embedded with time of request, requested slice type, downlink and uplink resource required denoted by $S_{id}^{(i)}$, $S_{type}^{(i)}$, $S_{DLreq}^{(i)}$ and $S_{ULreq}^{(i)}$, respectively. We consider that each request may need to create single/multiple slices at once. Thus the number of the requested slices is $S_{Nreq}^{(i)}$. The parameters $S_{STreq}^{(i)}$ denotes the service time requested by slice request i , and $S_{priority}^{(i)}$ denotes the priority of the slice request. We include the priority parameter so that every slice gets the chance of being considered in the optimization process and does not go unserved by the slice optimizer. A lower priority value means higher priority than the larger priority value. This prioritization model aligns with the priority-based scheduling (PBS) used in process scheduling in computing systems. For

example, if a slice request is of ‘0’ priority, it is the highest priority slice request to be served.

To give an example of a slice request, let us think that tenant ‘2’ wants three ‘eMBB’ (enhanced Mobile Broadband service) slices to be served, each of them having a DL and UL throughput required equal to 8 and 5 Gbps for 24 hours, with a requested priority of ‘0’. Then the slice request would look like as follows $S_{req}^{(2)} = (S_{id}^{(2)}, eMBB, 8, 5, 3, 24, 0)$. The slice id is an automated system-generated number such as a sequence number with embedded time of request to track the slice requests. Next, we define a slice pipeline as a queue that contains several slice requests from different tenants (different MVNOs). Thus, a slice pipeline is defined as $SP = \{S_{req}^{(1)}, S_{req}^{(2)}, \dots, S_{req}^{(N)}\}$, where we have N slice requests to be processed and thus, queued in the pipeline before applying the optimization algorithm. After applying the optimization, we divide the SP into currently serving slices set SP_{CS} , waiting to be served slices set SP_{queue} , and decommissioned slices set $SP_{disposal}$, where at any instance we update SP as, $SP = \{(SP_{CS} \cup SP_{queue}) \setminus SP_{disposal}\}$.

B. Slice Optimization Process Illustration

To illustrate the process we first divide it into several steps, as in Figure 1. Let us consider that at time $t = t_0$, the orchestrator receives a bunch of slice requests to be processed and served in the slice pipeline (SP) (marked as ①). When slice requests are waiting to serve/accept in the SP , the orchestrator triggers an event to run/initiate the optimization process. At time $t = t_0 + \Delta$, the optimization runs on the current slice pipeline (marked as ②). We assume that running the optimization itself takes another Δ time. Thus at time $t = t_0 + 2\Delta$; a set of requests are identified as profitable to serve and moved to currently serving slice set (SP_{CS}), while the rest of the requests are put in the waiting to serve queue SP_{queue} (marked as ③). We illustrate the mathematical optimization in the algorithm discussions in section II-C.

To ensure that the agreed upon slice requests are not left out from the optimization process, we apply the priority-based scheduling with aging technique in the SP_{queue} . PBS with aging is a well-known method and have diverse applications in the job or process scheduling ([6, 7]). The aging technique keeps increasing the priority of low-priority slice requests based on their waiting time to prevent starvation of any slices with low urgency. Here we assume that the orchestrator has no limitations on computing or processing capacity (considering the orchestrator has enough processing servers). Still, we have limitations on the available downlink and uplink throughput capacity. Thus, the orchestrator can process as many slice requests at once if the total requested UL and DL throughput are within the InfP’s UL and DL capacity limits.

Some of the slice requests currently serving may reach the end of the requested serving time and move to the decommissioning phase, thus leaving behind some additional unused capacity for the new slice requests to get admitted. For example, in the Figure 1, we find that the serving time of request two is less than the serving time of request five, and

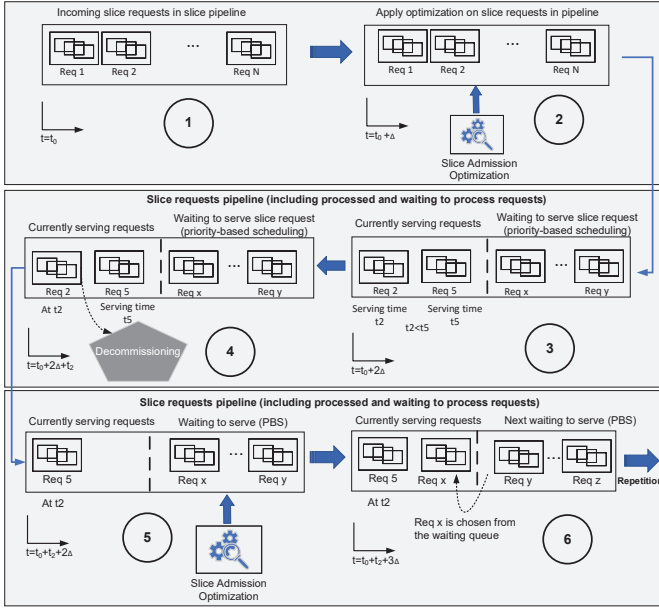


Fig. 1: Slice Admission Control Optimization Process Illustration

thus, request two moves to the decommissioning phase before request five. And whatever capacity was used by request two is now free and back to the infrastructure capacity for new slice requests. Thus, at time $t = t_0 + 2\Delta + t_2$, the orchestrator triggers another event that a slice gets decommissioned, and we need to rerun the optimization algorithm (marked as ④).

Next, at time $t = t_0 + 2\Delta + t_2$, the optimizer runs the algorithm on the requests waiting for queue and picks some of those requests that are profitable to serve as identified by the optimizer (marked as ⑤). Every time there is a change in the SP_{CS} , the optimizer runs. On the other hand, the SP_{queue} gets updated after every prespecified time (t_δ ; for example, after every $t_\delta = 15$ minutes). The new incoming slices are put into the SP_{queue} . Finally, the process goes on until there is another trigger to run the optimization algorithm, which means it waits until the next time either a slice request goes to decommissioning phase or some new slices arrive (marked as ⑥). Next we present the optimization formulation along with the algorithm.

C. Slice Optimization and Algorithm

Our objective in the optimization is to identify and serve profitable slices. At the same time, we want to minimize the opportunity cost. Also, we want to serve the agreed slice requests so that they are not left out while serving others. Thus, we form a multi-objective Knapsack problem. Furthermore, the optimization problem here is a bounded Knapsack problem because we can not divide the uplink and downlink throughput capacity requested in each slice request. Still, we can adjust the number of slices to serve within a slice request.

Let us consider, x_i is the number of slices to be accepted for the slice request $S_{req}^{(i)}$ as an output of the optimization (i.e., decision variables). Let, Pr_i be the per unit throughput (GBPS) price (in \$ value) which is a function

of the service request type ('eMBB', 'uRLLC', 'mMTC', etc.), i.e., $Pr_i = F(S_{type}^{(i)})$. Now, let us convert the slice request priority into a priority-weight using the expression $Pw_i = 1 - \frac{S_{priority}^{(i)}}{\sum_{j \in SP} S_{priority}^{(j)}}$. Pw_i ensures that we put more weight on the slice request with lower numerical priority value (For example, if a slice comes with '0' numeric priority value, it has the highest priority weight of 1.0). We took this approach to be align with the process scheduling in priority-based scheduling as found in the literature [8]. The objective functions and the constraints are given below.

Objective functions:

$$\begin{aligned} \max_{x_i} & \left\{ \sum_{i \in SP} x_i \times (S_{DLreq}^{(i)} + S_{ULreq}^{(i)}) \times Pr_i \times S_{STreq}^{(i)} \times Pw_i \right\} \\ \min_{x_i} & \left\{ (S_{Nreq}^{(i)} - x_i) \times Pr_i \times S_{STreq}^{(i)} \right\} \end{aligned} \quad (1)$$

Constraints (s.t.):

$$\begin{aligned} \sum_{i \in |SP|} S_{Nreq}^{(i)} \times S_{DLreq}^{(i)} & \leq InfPDL_{Cap}, \\ \sum_{i \in |SP|} S_{Nreq}^{(i)} \times S_{ULreq}^{(i)} & \leq InfPUL_{Cap}, \\ 0 \leq x_i \leq S_{Nreq}^{(i)} & \text{ and } x_i \text{ integers} \end{aligned}$$

The first expression in the Eq. (1) maximizes the revenue from the accepted slices. The second expression in the objective function minimizes the opportunity cost. The constraints that we have are the uplink and downlink throughput capacity constraint from infrastructure provider $InfPUL_{Cap}$, and $InfPDL_{Cap}$. In addition, the variable x_i can not surpass the number of slices requested in each slice request, and x_i is a positive integer quantity or '0'. We follow the algorithm 1 to optimize the slice admission requests. Here, we only consider limited throughput capacity, but we can add other parameters and constraints in the model as deemed necessary.

III. SIMULATION SETUP AND ANALYSIS FOR SLICE OPTIMIZATION

For the simulation, we take the following data as inputs. We assume that the price of uRLLC would be higher than mMTC/mIoT, and the price of mMTC would be higher than that of eMBB. We use Python PuLP library [9] to solve the optimization problem. The PuLP is an open-source linear programming (LP) package, which provides many industry-standard optimization solvers.

A. Simulation Input Parameters

For the optimization simulation, we have used the following data to find the cumulative revenue using the optimization algorithm.

- Infrastructure provider uplink/downlink throughput capacity = 1000 Gbps
- Price of eMBB, uRLLC (ultra-Reliable Low-Latency Communication), and mMTC (massive Machine Type

Algorithm 1: Profitable Slice Admission Algorithm

Result: Optimized slice request set
 $X_s = \{x_i | i \in SP \text{ and } x_i \neq 0\}$

Input: Slice pipeline SP and slice requests $SP_{req}^{(i)}$

Decision Variables: x_i denoting the allowed number of slices from $SP_{req}^{(i)}$

Initialization: $SP_{CS} \leftarrow \emptyset, SP_{queue} \leftarrow SP, SP_{disposal} \leftarrow \emptyset$

while $SP_{queue} \neq \emptyset$ **do**

Step 1: Find X_s by solving the optimization in Eq 1

Step 2: Update SP_{CS} and SP_{queue} , and $InfP$ capacity
 $SP_{CS} = \{SP_{req}^{(i)} | i \in SP_{queue} \text{ and } x_i \in X_s\}$
 $SP_{queue} = SP \setminus SP_{CS}$
 $InfPDL_{cap} = InfPDL_{cap} - \sum_{i \in X_s} S_{DLreq}^{(i)} \times x_i$
 $InfPUL_{cap} = InfPUL_{cap} - \sum_{i \in X_s} S_{ULreq}^{(i)} \times x_i$

Step 3: Apply PBS with aging on SP_{queue} every t_δ time

Step 4: Sort the slices in SP_{CS} according to the requested service time in descending order (i.e., $t_j < t_i$) format

Step 5: At t_j , the slice j left the network and the capacity becomes available, thus, update
 $InfPDL_{cap} += S_{DLreq}^{(j)} \times x_i,$
 $InfPUL_{cap} += S_{ULreq}^{(j)} \times x_i,$
 $SP_{disposal} = \{S_{req}^{(j)}\}$ and $SP_{CS} = SP_{CS} \setminus disposal$

Step 6: Repeat (until $SP_{queue} \neq \emptyset$)

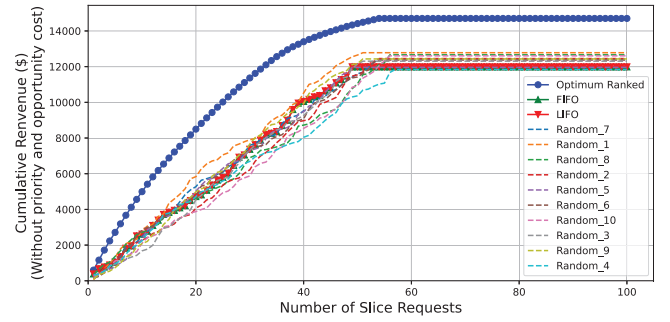
end

Communication service) slice per unit= \$5, \$10, \$8, respectively

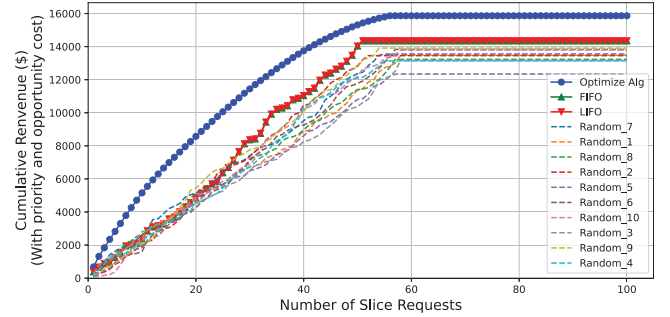
- Slice downlink and uplink throughputs are random numbers between (5, 10) Gbps and between (1, 10) Gbps, respectively
- Number of requested slices in each request is a random number between (1, 5), and priority of each request is a random number between (0, 50)
- Serving time of the slices in each request is set to 1 hour. The time is kept the same to understand the optimization performance without considering the queue performance, as we assumed that there is enough computing capacity for the orchestrator.

B. Simulation Results and Analysis

For this simulation, we have simulated two cases to see the affect of opportunity cost. The first case provides optimal results without considering the slice priority and opportunity cost as in Figure 2a. The second case shows the optimization output considering the slice priority and opportunity cost as given in Figure 2b. For each simulation, we compare the result of the optimization algorithm with that of FIFO (First In First Out), LIFO (Last In First Out), and ten random selections of the slice requests having satisfied the uplink and downlink throughput capacity constraints. We find that the proposed optimization algorithm provides maximum cumulative revenue over the accepted slice requests in both cases. The saturation point is there because even if we have 100 slice requests, but considering the uplink and downlink throughput capacity limitations of InfP, the optimizer can satisfy around 50 to 60 slice requests at once. The rest of the slices go into the queue



(a) Cumulative revenue without priority and opportunity cost



(b) Cumulative revenue with priority and opportunity cost

Fig. 2: Simulation result of slice optimization (cumulative revenue over the number of accepted slice requests).

and follow the PBS with the aging schedule. In the second case, considering priority and opportunity costs, we can get more cumulative revenue than in the first case. This finding shows that applying the optimization algorithm would yield more significant revenue in the long run than accepting slice requests as they arrive. Next, we present our dynamic slice admission control model using reinforcement learning.

IV. DYNAMIC SLICE ADMISSION CONTROL USING REINFORCEMENT LEARNING

To introduce dynamism in the slice admission control process, we propose a Markov decision process (MDP)-based model where an agent (i.e., the orchestrator) experiences different situations in the environment and learns to take actions based on the defined rewards and policies. We got some insights from work by Bakri et al. [4]. However, our novelty lies in the enhanced adjustments in the state descriptions and reward function for considering the opportunity cost and penalties for wrong decisions.

A. Slice System MDP Model

We model the slice requests as a Markov decision process (MDP) composed of four-tuples, $M = (S; A; T; R)$, where S is the set of system states, A is the set of actions, T is the transition probability from state S_t at time t to S_{t+1} at time $t + 1$ by taking action a_t . $r_t \in R$ is the reward obtained by performing the action a_t in the state S_t . The intelligent agent is an orchestrator, which takes admission decisions by learning from the environment. Including the

slice service types (SST) and their numbers at any given time, we model the state as a tuple of seven information elements. Thus a compound state is represented by $S = (S_{id}, n_{embb}, n_{uRLLC}, n_{mMTC}, n_{others}, b, st)$. Here, S_{id} is the state numeric identifier; n_{embb} , n_{uRLLC} , n_{mMTC} , n_{others} are the number of eMBB slices, uRLLC slices, mMTC/mIoT slices, and other slices that the orchestrator is serving; the b value encodes the slice type (SST), where $b=1,2,3,4$ represents eMBB, uRLLC, mMTC, and others, respectively. Finally, st represent the last action status (rejected / 0, accepted / 1, or left / 2). The action set is represented as follows:

$$a = \begin{cases} 0 & \text{if a new arrival slice request is rejected} \\ 1 & \text{if a new arrival slice request is accepted} \\ 2 & \text{if a slice left the environment (service time expiry)} \end{cases}$$

Reward Computations: The agent is motivated in the environment based on the reward it gets. We compute the reward using Eq. 2 as follows. There are several scenarios that we consider here. If the environment has the capacity but the agent rejects a slice request, it gets a penalty equivalent to the opportunity (revenue) that it misses. On the other hand, if there is no capacity and the agent rejects the slice, it receives a '0' reward. This situation is reflected in the Eq. 2 by considering the parameter λ_1 (reject scenario - line (a) in Eq.2). Now, if there is capacity and the agent accepts the slice request, it receives a positive reward; on the other hand, if there is no capacity, but the agent decided to accept the slice request, it gets a negative reward (or penalty). This condition is reflected by the parameter λ_2 (accept scenario - line (b) in Eq.2). Finally, if a slice leaves from the environment, the agent considers the opportunity (revenue) cost into the reward computation. There is always the chance to accept the lowest price slice request by allocating the free capacity. Thus we take the difference in the opportunity (revenue) lost and what we can get (left scenario - line (c) in Eq.2). The other parameters are the same as what we considered in the optimization model, for example, Pr_i and S_{STreq}^i indicate the per unit price and required service time of slice request i , respectively.

$$R_{InfP} = \begin{cases} \lambda_1 \times (S_{DLreq}^{(i)} + S_{ULreq}^{(i)}) \times Pr_i \times S_{STreq}^i & \dots(a) \\ \lambda_2 \times (S_{DLreq}^{(i)} + S_{ULreq}^{(i)}) \times Pr_i \times S_{STreq}^i & \dots(b) \\ (S_{DLreq}^{(i)} + S_{ULreq}^{(i)}) \times [\min(P_{eMBB}, P_{uRLLC}, P_{mMTC}, P_{others}) - Pr_i] \times S_{STreq}^i & \dots(c) \end{cases} \quad (2)$$

$$\text{sign}(i) = \text{sign}(C_{InfDL} - S_{DLreq}^{(i)}) \wedge \text{sign}(C_{InfUL} - S_{ULreq}^{(i)}) \quad (3)$$

$$\text{sign}(x - y) = \begin{cases} 1 & \text{if } x \leq y \\ -1 & \text{otherwise} \end{cases} \quad (4)$$

$$\lambda_1 = \begin{cases} 0 & \text{if } \text{sign}(i) = -1 \\ -1 & \text{if } \text{sign}(i) = +1 \end{cases} \quad (5)$$

$$\lambda_2 = \begin{cases} 1 & \text{if } \text{sign}(i) = 1 \\ -1 & \text{if } \text{sign}(i) = -1 \end{cases} \quad (6)$$

We have used several reinforcement learning techniques to compare the performance and discuss those in the below sections.

B. Q-Learning, SARSA, Expected SARSA, and DQN

Q-Learning and SARSA algorithms are Reinforcement Learning techniques that use Temporal Difference (TD) update to improve the agent's behavior. The "Expected SARSA" technique is an alternative for improving the agent's policy. It is very similar to SARSA and Q-Learning but differs in the action value function it follows. SARSA is an on-policy technique, Q-learning is an off-policy technique, but Expected SARSA can be used either as an on-policy or off-policy. The basic idea behind Q-Learning is to use the Bellman optimality equation as an iterative update. Thus, in the Q-Learning, the Q-table is updated as follows.

$$Q(S_{t+1}, A_{t+1}) \leftarrow (1 - \alpha)Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a \in A} Q(S_{t+1}, a) \right] \quad (7)$$

where,

- $Q(S_t, A_t)$ and $Q(S_{t+1}, A_{t+1})$ is the old and new Q value
- α is the learning rate
- γ is the discount factor to take the future rewards into considerations
- $\max_{a \in A} Q(S_{t+1}, a)$ is the estimation of the optimal future action from the available actions

In the SARSA and Expected SARSA, the Q-table update is done as in Eq. 8 and Eq. 9, respectively.

$$Q(S_{t+1}, A_{t+1}) \leftarrow (1 - \alpha)Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) \right] \quad (8)$$

$$Q(S_{t+1}, A_{t+1}) \leftarrow (1 - \alpha)Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \left[\mathbb{E}(Q(S_{t+1}, A_{t+1}) | S_{t+1}) \right] \right] \quad (9)$$

It is impractical to represent the Q-function as a table containing values for each combination of S and A because the table gets larger. Thus as an alternative, we train a function approximator (i.e., a neural network with parameters θ) to estimate the Q-values, i.e. $Q(s, a, \theta) \approx Q^*(s, a)$. This is known as Deep Q-Learning or Deep Q-Network (DQN). In DQN, we minimizing the following loss function at each step i :

$$L_i(\theta_i) = \mathbb{E}_{s,a,r,s_{t+1} \sim \rho(\cdot)} [y_i - Q(s, a; \theta_i)^2] \quad (10)$$

where, $y_i = r_t + \gamma \max_{a \in A} Q(s_{t+1}, a_{t+1}; \theta_{i-1})$

Here, y_i is the TD (temporal difference) target, and $y_i - Q$ is the TD error. ρ represents the behaviour distribution, the distribution over transitions (s, a, r, s_{t+1}) collected from the environment.

V. SIMULATION RESULTS AND ANALYSIS FOR RL MODEL

We create a slice environment in Python for the simulation following the OpenAI GYM environment. We then simulate the four algorithms with a learning rate = 0.01 as given in Figure 3. We found that DQN provides much better cumulative rewards (i.e., cumulative revenue) than the other algorithms (Q-Learning, SARSA, Expected SARSA). We also tested the DQN model by varying the discount factor parameter and found that $\gamma = 0.99$ provides more rewards than the other discount factor values as shown in Figure 4. The only problem with the DQN (similar to other deep neural networks) is that it is comparatively slow and takes a long time to perform the simulation (around 45~60 minutes for 100 episodes in 32GB RAM in a Windows 10 machine). But using high-performance computing systems (such as graphics processing unit or GPUs), we can overcome the simulation time problem.

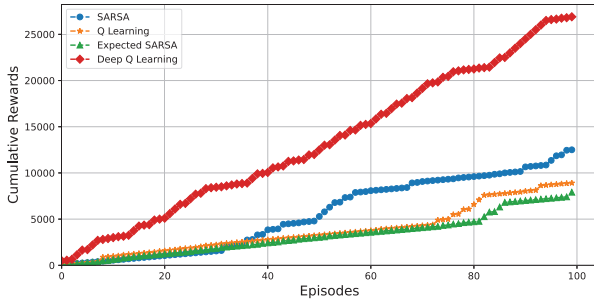


Fig. 3: Slice admission control simulation output using RL algorithms - cumulative rewards over episodes

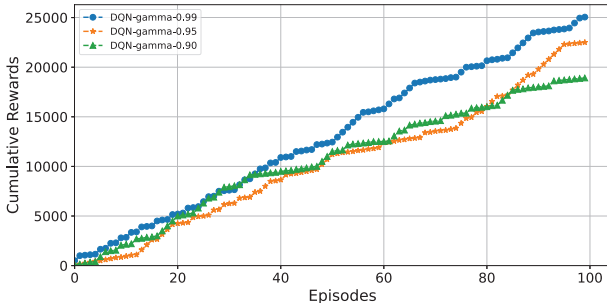


Fig. 4: DQN simulation for different values of discount factor hyperparameter ($\gamma = 0.99, 0.95, 0.90$)

VI. CONCLUSION AND FUTURE PLAN

This work addresses the challenges of dynamic slice admission control in the 5G telecommunication network. First, we proposed a multi-objective optimization algorithm and priority-based scheduling technique to maximize the infrastructure provider's long-term revenue. 5G network orchestrators can implement this optimization algorithm to automate the admission process. Second, we proposed an MDP model to accept or reject decisions on incoming slice requests dynamically. We tested several reinforcement learning algorithms such as Q-Learning, Deep Q-Network, SARSA, and Expected SARSA to compare the performance of the algorithms on this problem. We found that DQN performs better than the other algorithms. In the future, we plan to tune the hyperparameters for DQN, combine both modeling techniques in a unified model, and validate the model using a 5G test environment.

REFERENCES

- [1] 3GPP, "System architecture for the 5G System (5GS) (3GPP TS 23.501 version 16.6.0 Release 16)," 10 2020, version 14.2.2.
- [2] P. Rost, C. Mannweiler, D. S. Michalopoulos, C. Sartori, V. Sciancalepore, N. Sastry, O. Holland, S. Tayade, B. Han, D. Bega *et al.*, "Network slicing to enable scalability and flexibility in 5g mobile networks," *IEEE Communications magazine*, vol. 55, no. 5, pp. 72–79, 2017.
- [3] B. Han, S. Tayade, and H. D. Schotten, "Modeling profit of sliced 5g networks for advanced network resource management and slice implementation," in *2017 IEEE symposium on computers and communications (ISCC)*. IEEE, 2017, pp. 576–581.
- [4] S. Bakri, B. Brik, and A. Ksentini, "On using reinforcement learning for network slice admission control in 5g: Offline vs. online," *International Journal of Communication Systems*, vol. 34, no. 7, p. e4757, 2021.
- [5] B. Han, A. DeDomenico, G. Dandachi, A. Drosou, D. Tzovaras, R. Querio, F. Moggio, O. Bulakci, and H. D. Schotten, "Admission and congestion control for 5g network slicing," in *2018 IEEE Conference on Standards for Communications and Networking (CSCN)*. IEEE, 2018, pp. 1–6.
- [6] O. Mahitha and V. Suma, "Pre-emptive priority scheduling with aging technique for effective disaster management in cloud," *Graduate Research in Engineering and Technology (GRET): An International Journal*, vol. 1, no. 2, 2013.
- [7] M. Balajee, B. Suresh, M. Suneetha, V. V. Rani, and G. Veerajay, "Preemptive job scheduling with priorities and starvation cum congestion avoidance in clusters," in *2010 Second International Conference on Machine Learning and Computing*. IEEE, 2010, pp. 255–259.
- [8] M. Wolf, *Computers as components: principles of embedded computing system design*. Elsevier, 2012.
- [9] S. Mitchell, M. OSullivan, and I. Dunning, "Pulp: a linear programming toolkit for python," *The University of Auckland, Auckland, New Zealand*, p. 65, 2011.